

DATA TRANSFORMER

Advanced SQL Analytics, Relational Joins, Window Functions, CASE Expressions & String/Date Transformations

Student Name	Priyal Patel
Course & Subject	SQL & Advanced Relational Database Management System (RDBMS)
DBMS Platform	MySQL Server & phpMyAdmin (Apache / XAMPP)
GitHub Repository	github.com/priyal-24/Data-Transformer

Executive Summary & Overview

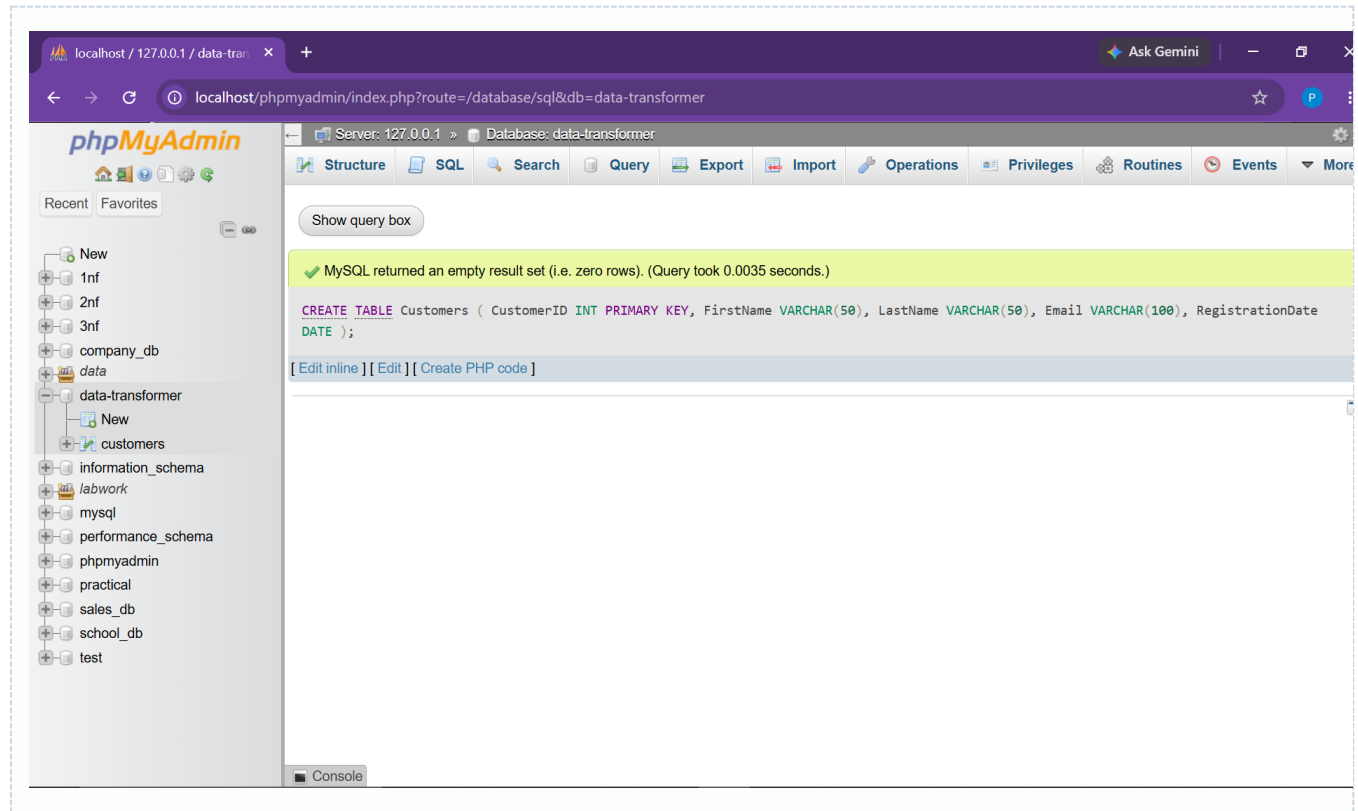
Data Transformer is a comprehensive SQL project designed to simulate a corporate relational database architecture and execute advanced analytical operations. The schema connects three primary entities: Customers, Orders, and Employees. This practical implementation report documents the complete database setup and the execution of 17 core analytical queries covering Relational JOINS (INNER, LEFT, RIGHT, FULL OUTER via UNION), Correlated & Nested Subqueries, Date & String manipulation functions, Window Functions (SUM OVER, RANK OVER), and dynamic Conditional Logic using SQL CASE expressions.

Each subsequent page of this report presents an individual SQL task with its exact query implementation and the corresponding execution output result captured directly from MySQL / phpMyAdmin.

1. DATABASE SCHEMA & TABLE CREATION (DDL)

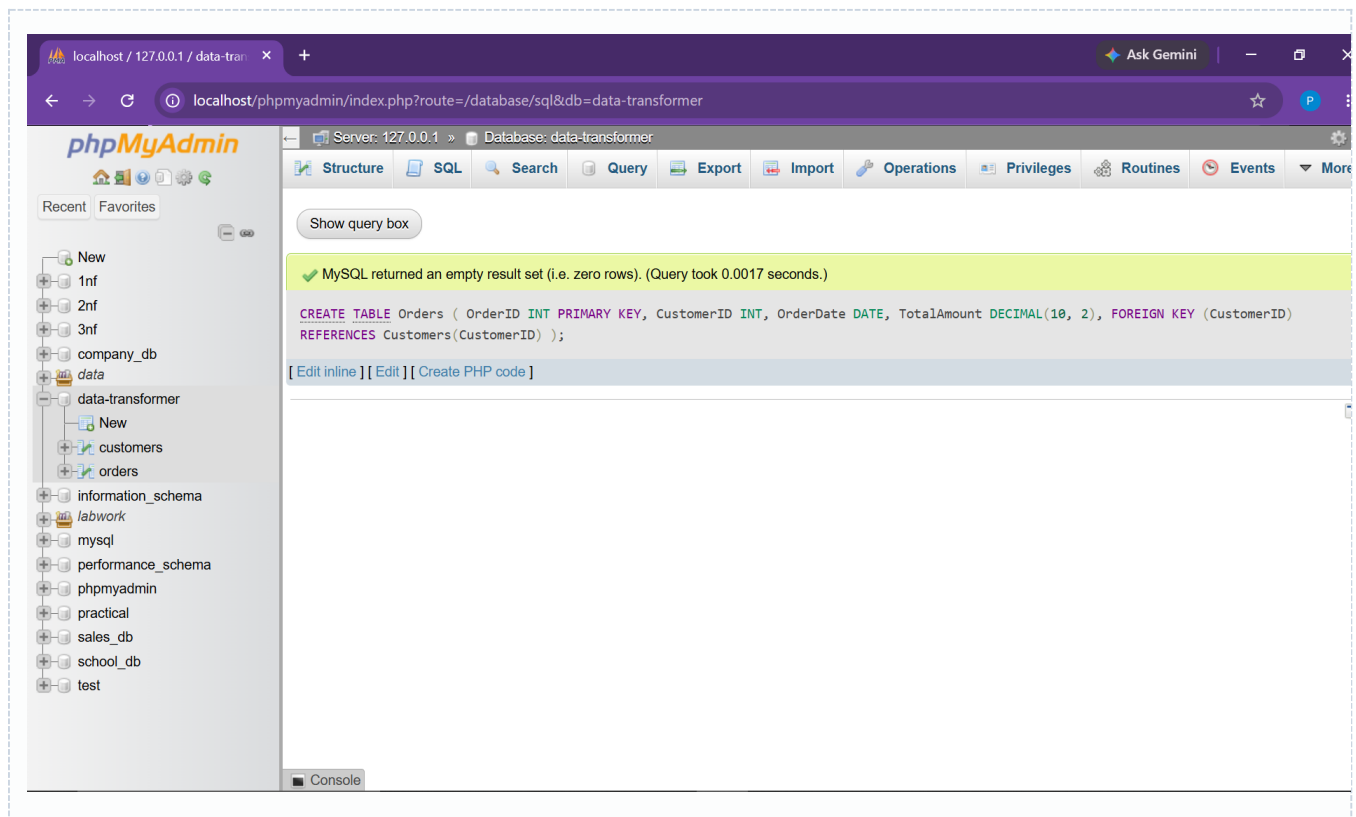
• 1.1 Create Customers Table

```
CREATE TABLE Customers (  
    CustomerID INT PRIMARY KEY,  
    FirstName VARCHAR(50),  
    LastName VARCHAR(50),  
    Email VARCHAR(100),  
    RegistrationDate DATE  
);
```



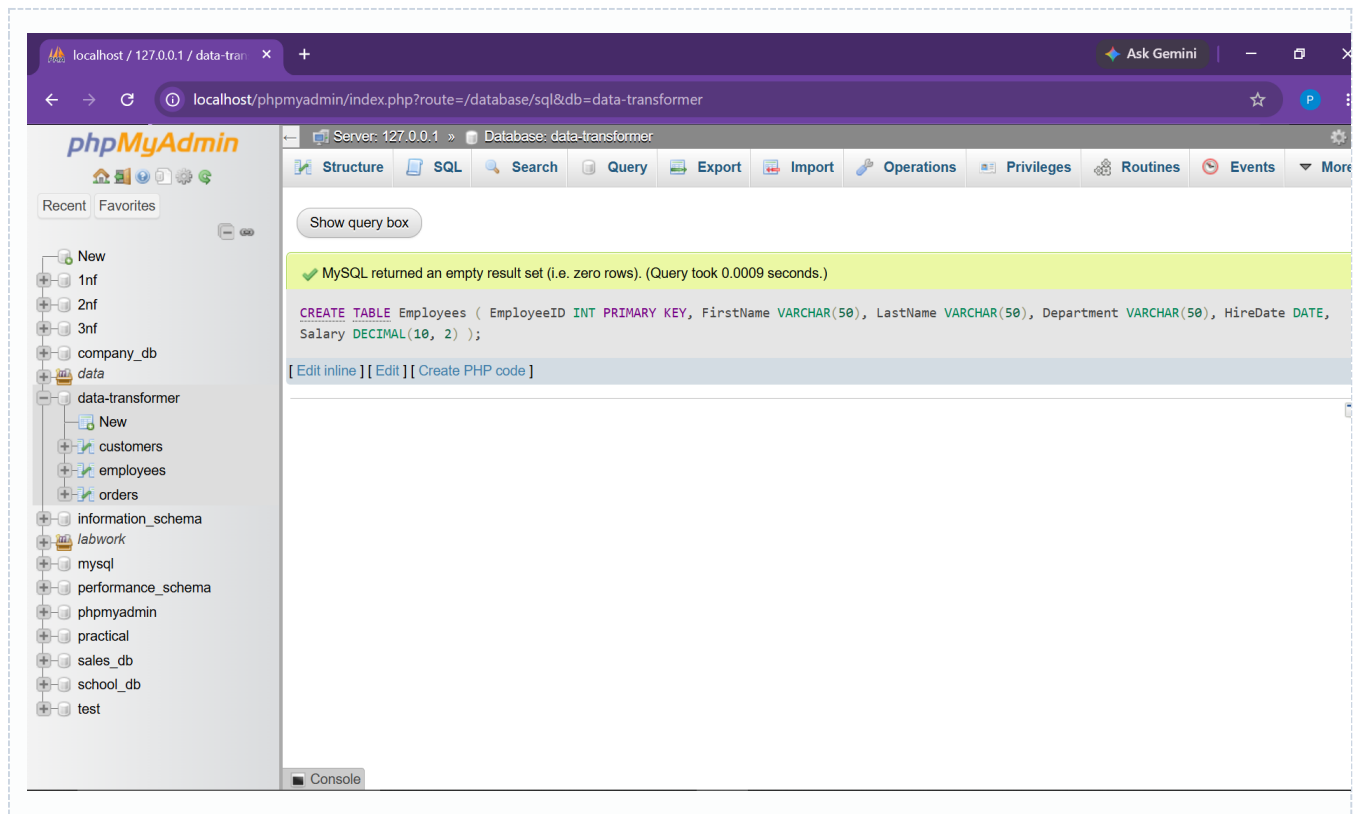
• 1.2 Create Orders Table with Foreign Key

```
CREATE TABLE Orders (  
    OrderID INT PRIMARY KEY,  
    CustomerID INT,  
    OrderDate DATE,  
    TotalAmount DECIMAL(10, 2),  
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)  
);
```



• 1.3 Create Employees Table

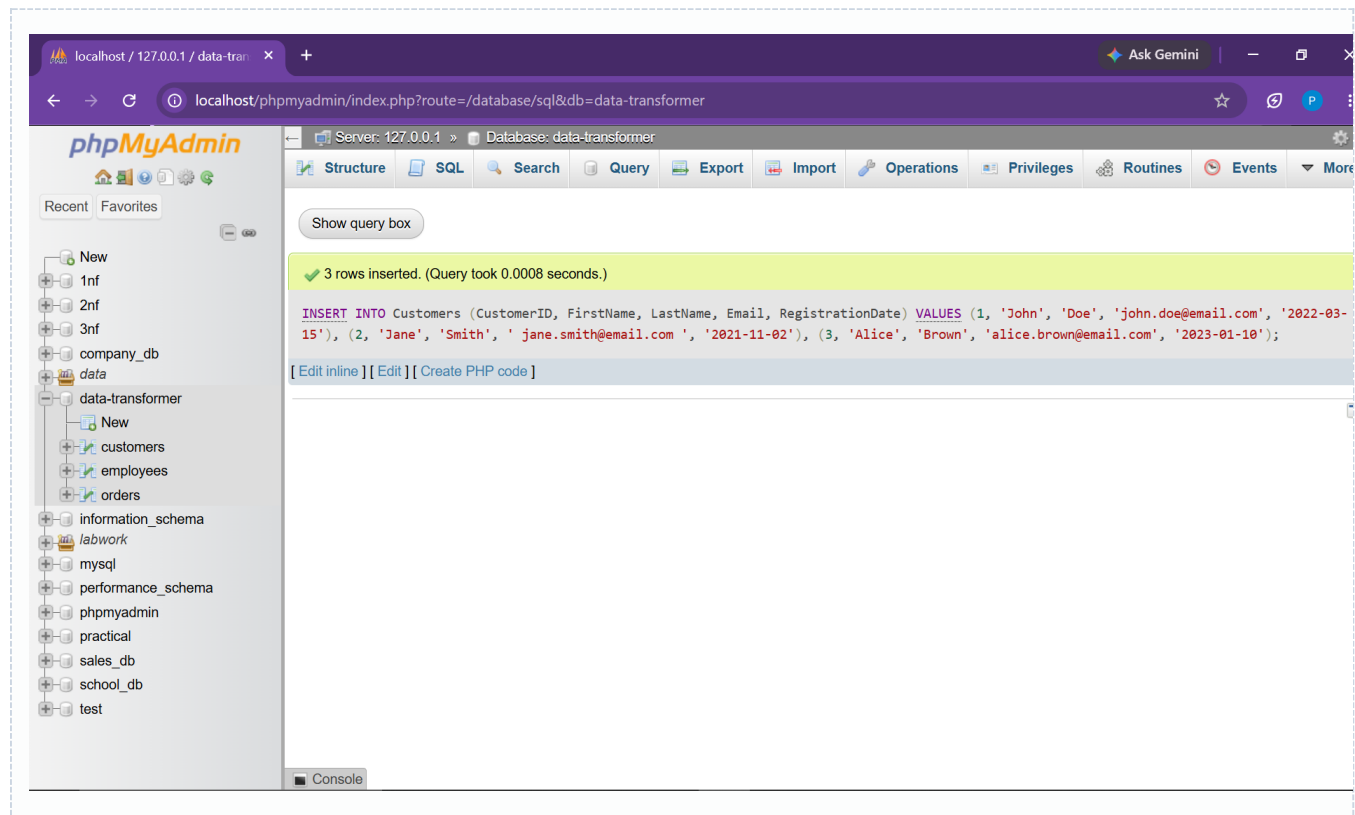
```
CREATE TABLE Employees (  
    EmployeeID INT PRIMARY KEY,  
    FirstName VARCHAR(50),  
    LastName VARCHAR(50),  
    Department VARCHAR(50),  
    HireDate DATE,  
    Salary DECIMAL(10, 2)  
);
```



2. SAMPLE DATA SEEDING (DML)

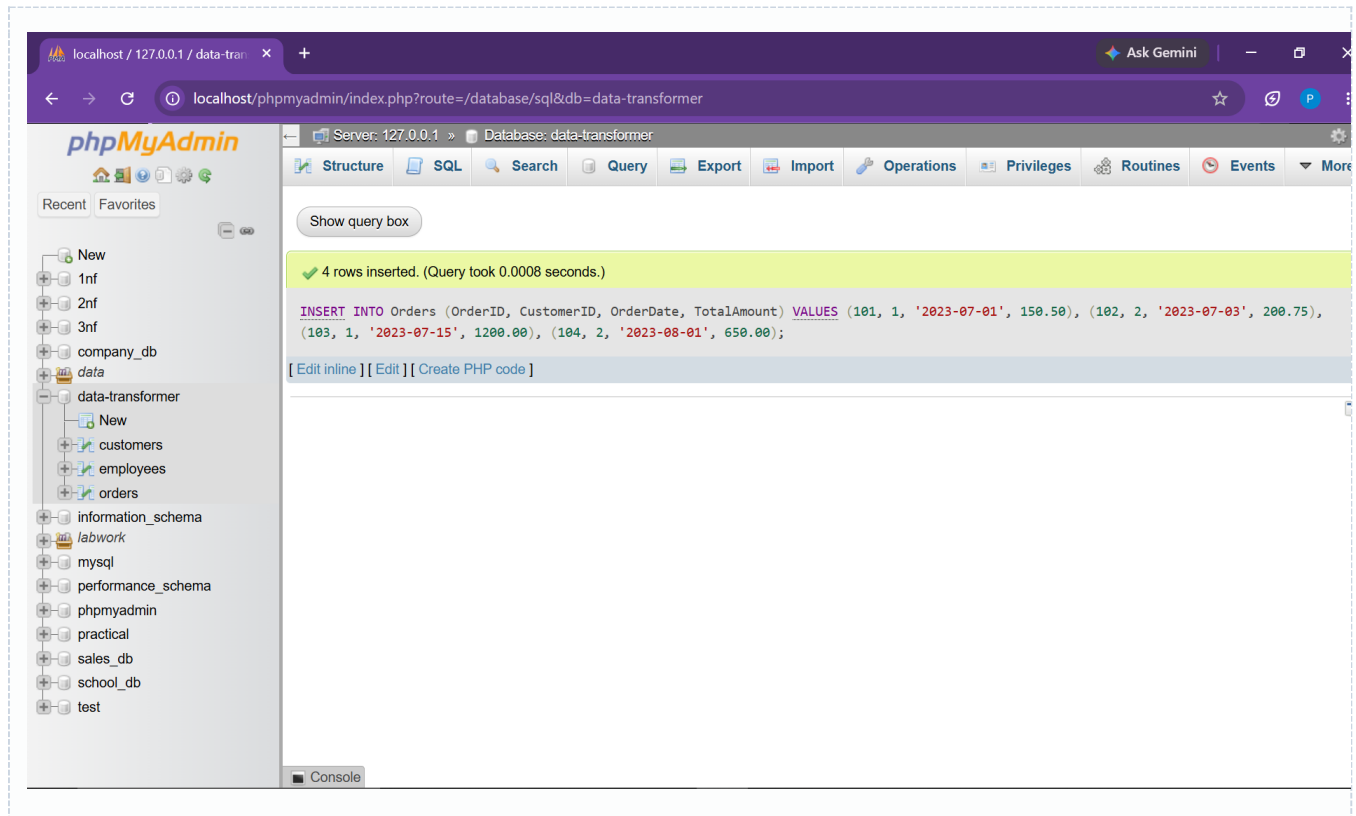
• 2.1 Insert Records into Customers Table

```
INSERT INTO Customers (CustomerID, FirstName, LastName, Email, RegistrationDate) VALUES
(1, 'John', 'Doe', 'john.doe@email.com', '2022-03-15'),
(2, 'Jane', 'Smith', 'jane.smith@email.com', '2021-11-02'),
(3, 'Alice', 'Brown', 'alice.brown@email.com', '2023-01-10');
```



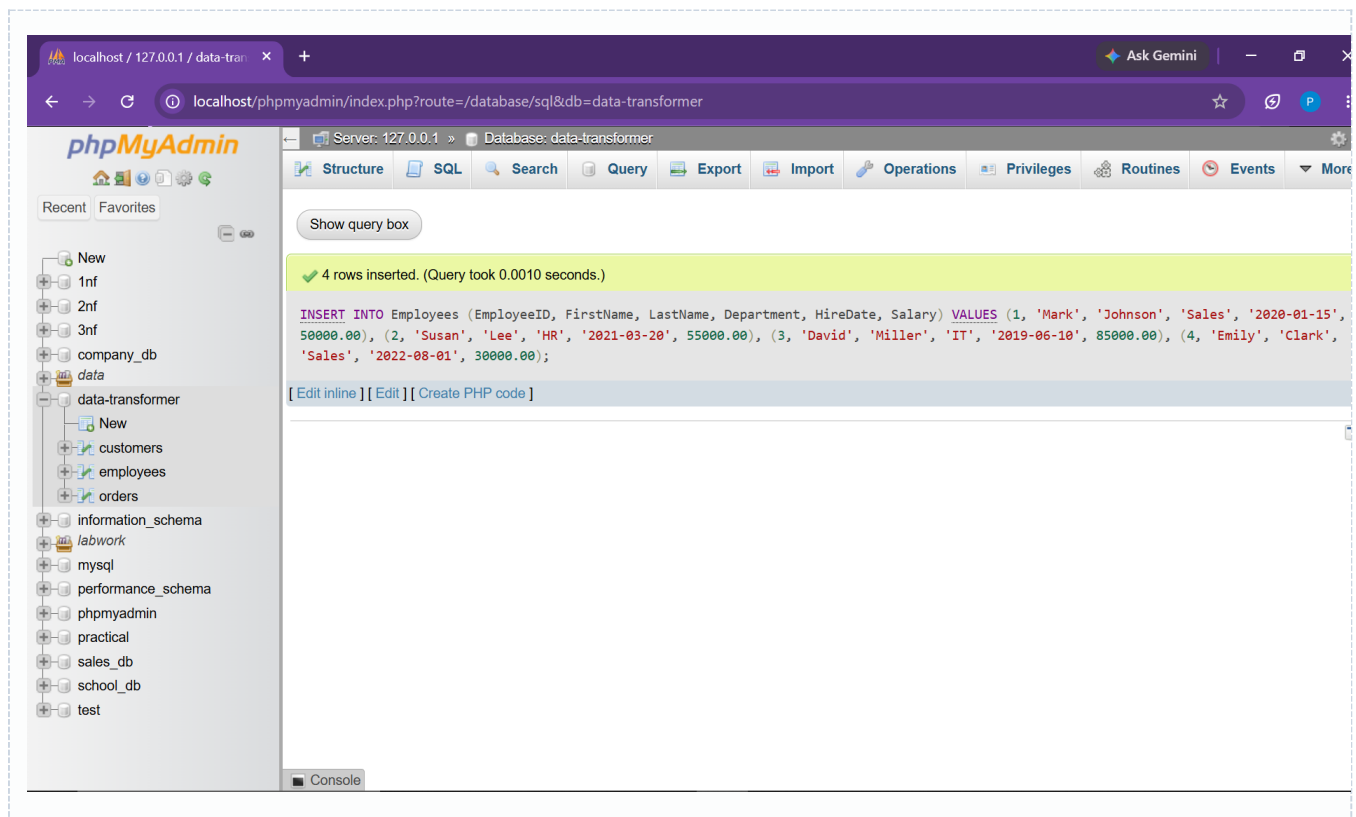
• 2.2 Insert Records into Orders Table

```
INSERT INTO Orders (OrderID, CustomerID, OrderDate, TotalAmount) VALUES
(101, 1, '2023-07-01', 150.50),
(102, 2, '2023-07-03', 200.75),
(103, 1, '2023-07-15', 1200.00),
(104, 2, '2023-08-01', 650.00);
```



• 2.3 Insert Records into Employees Table

```
INSERT INTO Employees (EmployeeID, FirstName, LastName, Department, HireDate, Salary) VALUES
(1, 'Mark', 'Johnson', 'Sales', '2020-01-15', 50000.00),
(2, 'Susan', 'Lee', 'HR', '2021-03-20', 55000.00),
(3, 'David', 'Miller', 'IT', '2019-06-10', 85000.00),
(4, 'Emily', 'Clark', 'Sales', '2022-08-01', 30000.00);
```



3. CORE ANALYTICAL QUERIES & TRANSFORMATIONS

- **Task.1 INNER JOIN: Retrieve all orders and customer details where orders exist**

Objective:

Retrieve transaction records merged with customer information where a valid relationship exists between both tables.

```
SELECT
    o.OrderID,
    o.OrderDate,
    o.TotalAmount,
    c.CustomerID,
    c.FirstName,
    c.LastName,
    c.Email
FROM Orders o
INNER JOIN Customers c ON o.CustomerID = c.CustomerID;
```

The screenshot shows the phpMyAdmin interface for a database named 'data-transformer'. The left sidebar displays a tree view of databases and tables, including 'data-transformer' with tables 'customers', 'employees', and 'orders'. The main panel shows the 'Query' tab with the following SQL query:

```
SELECT o.OrderID, o.OrderDate, o.TotalAmount, c.CustomerID, c.FirstName, c.LastName, c.Email FROM Orders o INNER JOIN Customers c ON o.CustomerID = c.CustomerID;
```

The query results are displayed in a table with 4 rows. The table has columns: OrderID, OrderDate, TotalAmount, CustomerID, FirstName, LastName, and Email. The results are:

OrderID	OrderDate	TotalAmount	CustomerID	FirstName	LastName	Email
101	2023-07-01	150.50	1	John	Doe	john.doe@email.com
102	2023-07-03	200.75	2	Jane	Smith	jane.smith@email.com
103	2023-07-15	1200.00	1	John	Doe	john.doe@email.com
104	2023-08-01	650.00	2	Jane	Smith	jane.smith@email.com

Observation:

Successfully returned 4 rows. Customer 3 ('Alice Brown') is excluded because she has not placed any orders yet.

- **Task.2 LEFT JOIN: Retrieve all customers and their corresponding orders (if any)**

Objective:

Retrieve all registered customers regardless of order history, populating order columns with NULL for non-purchasing customers.

```
SELECT
    c.CustomerID,
    c.FirstName,
    c.LastName,
    o.OrderID,
    o.OrderDate,
    o.TotalAmount
FROM Customers c
LEFT JOIN Orders o ON c.CustomerID = o.CustomerID;
```

The screenshot shows the phpMyAdmin interface for a database named 'data-transformer'. The left sidebar displays a tree view of databases and tables, including 'data-transformer' with tables 'customers', 'employees', and 'orders'. The main panel shows the 'Query' tab with the following SQL query:

```
SELECT c.CustomerID, c.FirstName, c.LastName, o.OrderID, o.OrderDate, o.TotalAmount FROM Customers c LEFT JOIN Orders o ON c.CustomerID = o.CustomerID;
```

Below the query, a message states: "Showing rows 0 - 4 (5 total, Query took 0.0021 seconds.)". The results are displayed in a table with 5 rows:

CustomerID	FirstName	LastName	OrderID	OrderDate	TotalAmount
1	John	Doe	101	2023-07-01	150.50
2	Jane	Smith	102	2023-07-03	200.75
1	John	Doe	103	2023-07-15	1200.00
2	Jane	Smith	104	2023-08-01	650.00
3	Alice	Brown	NULL	NULL	NULL

Observation:

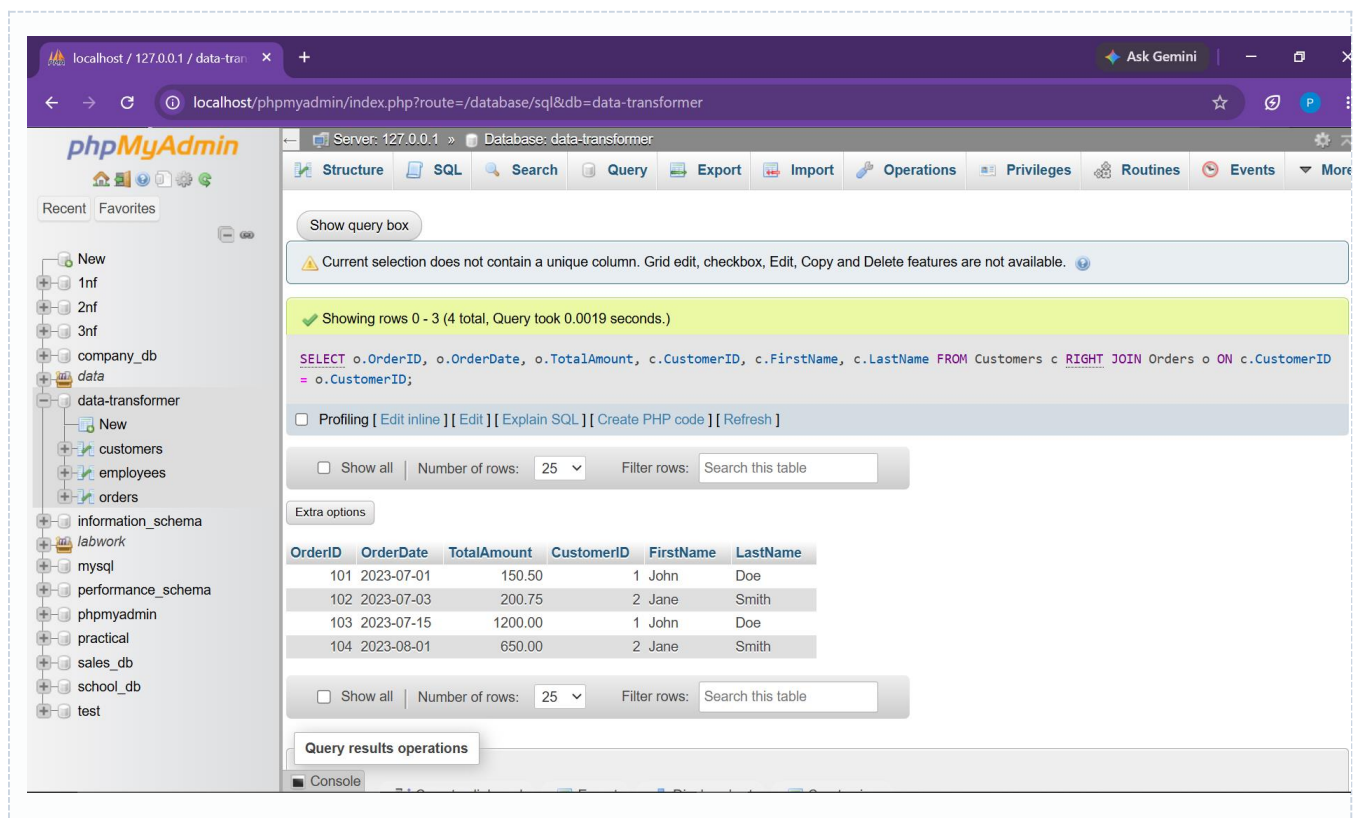
Returned 5 rows. Customers 1 and 2 display their corresponding orders, while Customer 3 displays NULL across all order attributes.

- **Task.3 RIGHT JOIN: Retrieve all orders and their corresponding customers (if any)**

Objective:

Preserve all order ledger rows while fetching matching customer master records.

```
SELECT
    o.OrderID,
    o.OrderDate,
    o.TotalAmount,
    c.CustomerID,
    c.FirstName,
    c.LastName
FROM Customers c
RIGHT JOIN Orders o ON c.CustomerID = o.CustomerID;
```



The screenshot shows the phpMyAdmin web interface. The left sidebar displays a database structure tree with 'data-transformer' selected, containing tables 'customers', 'employees', and 'orders'. The main panel shows the 'Query' tab with the following SQL query:

```
SELECT o.OrderID, o.OrderDate, o.TotalAmount, c.CustomerID, c.FirstName, c.LastName FROM Customers c RIGHT JOIN Orders o ON c.CustomerID = o.CustomerID;
```

The query results are displayed in a table with 4 rows. The table has columns: OrderID, OrderDate, TotalAmount, CustomerID, FirstName, and LastName.

OrderID	OrderDate	TotalAmount	CustomerID	FirstName	LastName
101	2023-07-01	150.50	1	John	Doe
102	2023-07-03	200.75	2	Jane	Smith
103	2023-07-15	1200.00	1	John	Doe
104	2023-08-01	650.00	2	Jane	Smith

Observation:

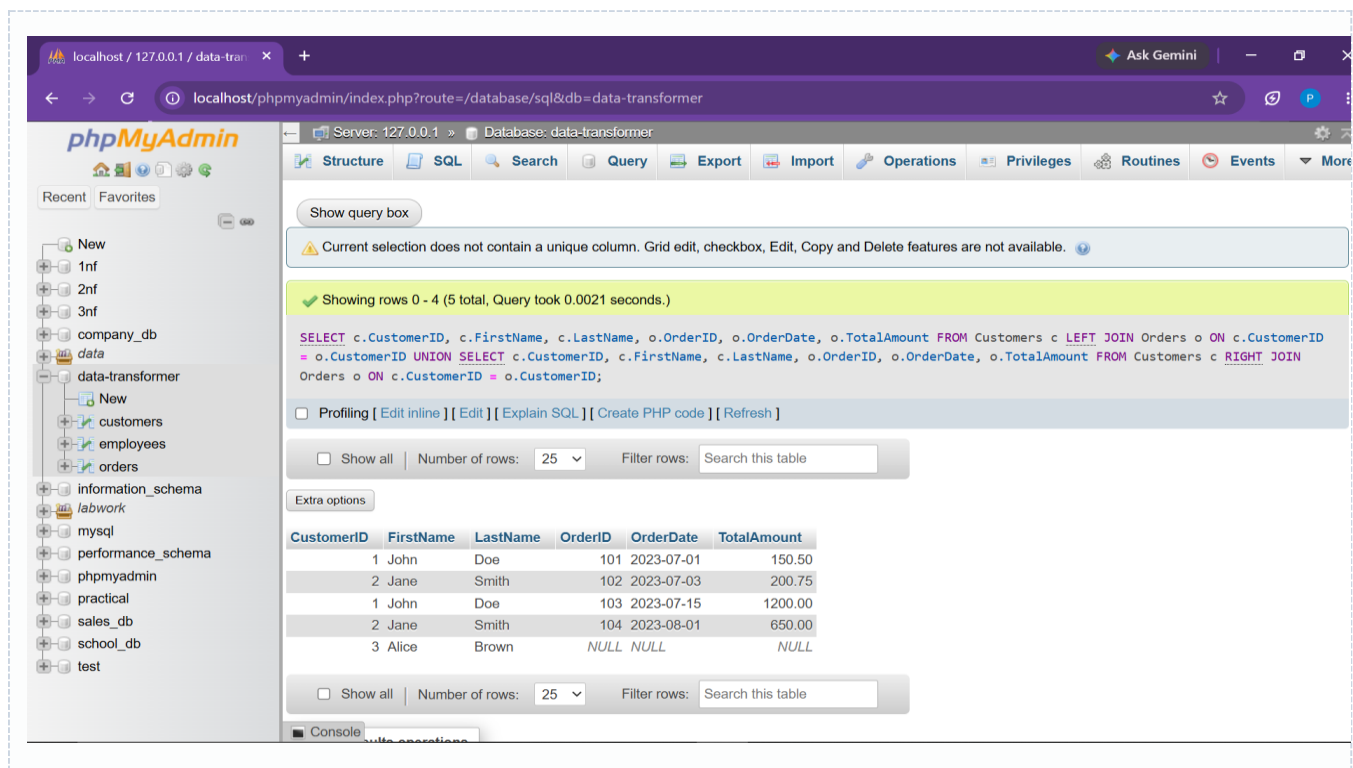
Successfully returned 4 rows, ensuring complete preservation of transactional sales ledger entries.

- **Task.4 FULL OUTER JOIN: Retrieve all customers and all orders (Simulated using UNION)**

Objective:

Simulate Full Outer Join via UNION of LEFT JOIN and RIGHT JOIN to map all records from both entities.

```
SELECT
    c.CustomerID,
    c.FirstName,
    c.LastName,
    o.OrderID,
    o.OrderDate,
    o.TotalAmount
FROM Customers c
LEFT JOIN Orders o ON c.CustomerID = o.CustomerID
UNION
SELECT
    c.CustomerID,
    c.FirstName,
    c.LastName,
    o.OrderID,
    o.OrderDate,
    o.TotalAmount
FROM Customers c
RIGHT JOIN Orders o ON c.CustomerID = o.CustomerID;
```



The screenshot displays the phpMyAdmin interface for a database named 'data-transformer'. The left sidebar shows the database structure, including tables 'customers' and 'orders'. The main panel shows the results of a SQL query. The query is a UNION of a LEFT JOIN and a RIGHT JOIN between the 'Customers' and 'Orders' tables. The results table contains 5 rows, showing the unified data from both tables. The last row represents a customer with no orders, indicated by NULL values for OrderID, OrderDate, and TotalAmount.

CustomerID	FirstName	LastName	OrderID	OrderDate	TotalAmount
1	John	Doe	101	2023-07-01	150.50
2	Jane	Smith	102	2023-07-03	200.75
1	John	Doe	103	2023-07-15	1200.00
2	Jane	Smith	104	2023-08-01	650.00
3	Alice	Brown	NULL	NULL	NULL

Observation:

Returned 5 distinct rows representing the unified outer join dataset.

- **Task.5 Subquery to find customers who placed orders worth more than the average amount**

Objective:

Filter customers who have placed at least one individual order greater than the aggregate average order value (\$550.31).

```
SELECT DISTINCT
    c.CustomerID,
    c.FirstName,
    c.LastName
FROM Customers c
JOIN Orders o ON c.CustomerID = o.CustomerID
WHERE o.TotalAmount > (
    SELECT AVG(TotalAmount) FROM Orders
);
```

The screenshot shows the phpMyAdmin interface in a web browser. The left sidebar displays a database structure with a tree view. The main panel shows the 'Table: Customers' view. A SQL query is entered in the 'Show query box' area, and the results are displayed below. The query is: `SELECT DISTINCT c.CustomerID, c.FirstName, c.LastName FROM Customers c JOIN Orders o ON c.CustomerID = o.CustomerID WHERE o.TotalAmount > ( SELECT AVG(TotalAmount) FROM Orders );`. The results show 2 rows: CustomerID 1 (John Doe) and CustomerID 2 (Jane Smith). The interface includes various navigation and tool buttons at the top and bottom.

CustomerID	FirstName	LastName
1	John	Doe
2	Jane	Smith

Observation:

Successfully returned 2 distinct customers: John Doe (Order 103 = \$1200.00) and Jane Smith (Order 104 = \$650.00).

- **Task.6 Subquery to find employees with salaries above the average salary**

Objective:

Identify employee records whose compensation exceeds the organization's average payroll figure (\$55,000.00).

```
SELECT
    EmployeeID,
    FirstName,
    LastName,
    Department,
    Salary
FROM Employees
WHERE Salary > (
    SELECT AVG(Salary) FROM Employees
);
```

The screenshot shows the phpMyAdmin web interface in a browser. The URL is localhost/phpmyadmin/index.php?route=/table/sql&db=data-transformer&table=employees. The interface displays the 'Employees' table structure and the results of the following SQL query:

```
SELECT EmployeeID, FirstName, LastName, Department, Salary FROM Employees WHERE Salary > ( SELECT AVG(Salary) FROM Employees );
```

The query results show 1 record: David Miller (IT Department, Salary: \$85,000.00). The interface includes a sidebar with a database tree, a top navigation bar with various tools (Browse, Structure, SQL, Search, Insert, Export, Import, Privileges, Operations, Tracking, More), and a bottom section for query results operations (Print, Copy to clipboard, Export, Display chart, Create view).

EmployeeID	FirstName	LastName	Department	Salary
3	David	Miller	IT	85000.00

Observation:

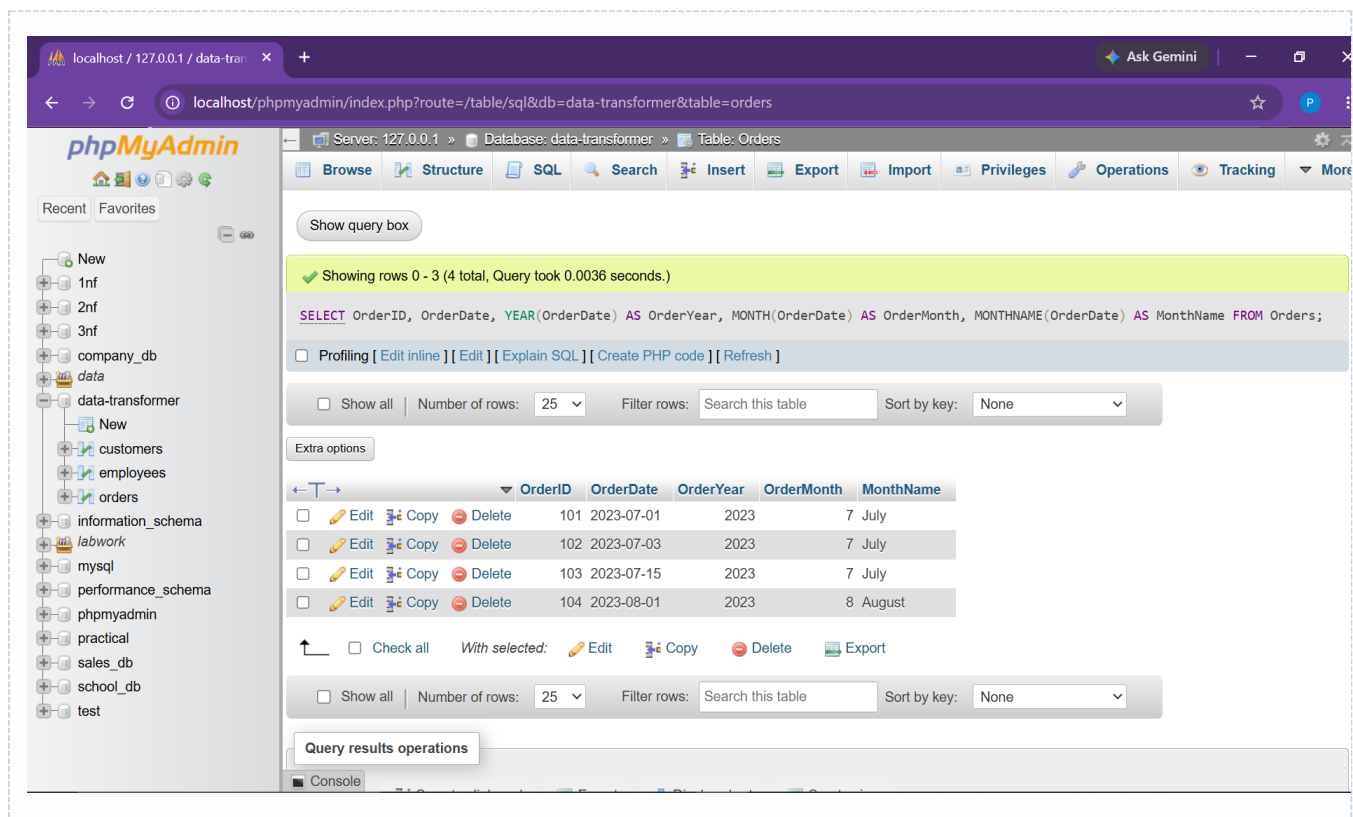
Returned 1 record: David Miller (IT Department, Salary: \$85,000.00).

- **Task.7** Extract the year and month from the OrderDate

Objective:

Deconstruct transaction timestamps into Year, Month numeric index, and readable Month name.

```
SELECT
    OrderID,
    OrderDate,
    YEAR(OrderDate) AS OrderYear,
    MONTH(OrderDate) AS OrderMonth,
    MONTHNAME(OrderDate) AS MonthName
FROM Orders;
```



The screenshot shows the phpMyAdmin interface for a database named 'data-transformer'. The 'Orders' table is selected. The SQL query executed is: `SELECT OrderID, OrderDate, YEAR(OrderDate) AS OrderYear, MONTH(OrderDate) AS OrderMonth, MONTHNAME(OrderDate) AS MonthName FROM Orders;`. The results show 4 rows of data, displaying the OrderID, OrderDate, and the extracted Year, Month, and MonthName.

OrderID	OrderDate	OrderYear	OrderMonth	MonthName
101	2023-07-01	2023	7	July
102	2023-07-03	2023	7	July
103	2023-07-15	2023	7	July
104	2023-08-01	2023	8	August

Observation:

Returned 4 rows displaying extracted temporal components (Year 2023, Months 7-July and 8-August).

- **Task.8** Calculate the difference in days between two dates (order date and current date)

Objective:

Compute the elapsed interval in days between each transaction's OrderDate and CURRENT_DATE().

```
SELECT
    OrderID,
    OrderDate,
    CURRENT_DATE() AS CurrentDate,
    DATEDIFF(CURRENT_DATE(), OrderDate) AS DaysDifference
FROM Orders;
```

The screenshot shows the phpMyAdmin interface for a database named 'data-transformer'. The 'Orders' table is selected. A SQL query is executed, showing 4 rows of results. The query calculates the difference in days between the 'OrderDate' and the current date ('CURRENT_DATE()').

Query executed:

```
SELECT OrderID, OrderDate, CURRENT_DATE() AS CurrentDate, DATEDIFF(CURRENT_DATE(), OrderDate) AS DaysDifference FROM Orders;
```

Results:

OrderID	OrderDate	CurrentDate	DaysDifference
101	2023-07-01	2026-10-05	1192
102	2023-07-03	2026-10-05	1190
103	2023-07-15	2026-10-05	1178
104	2023-08-01	2026-10-05	1161

Observation:

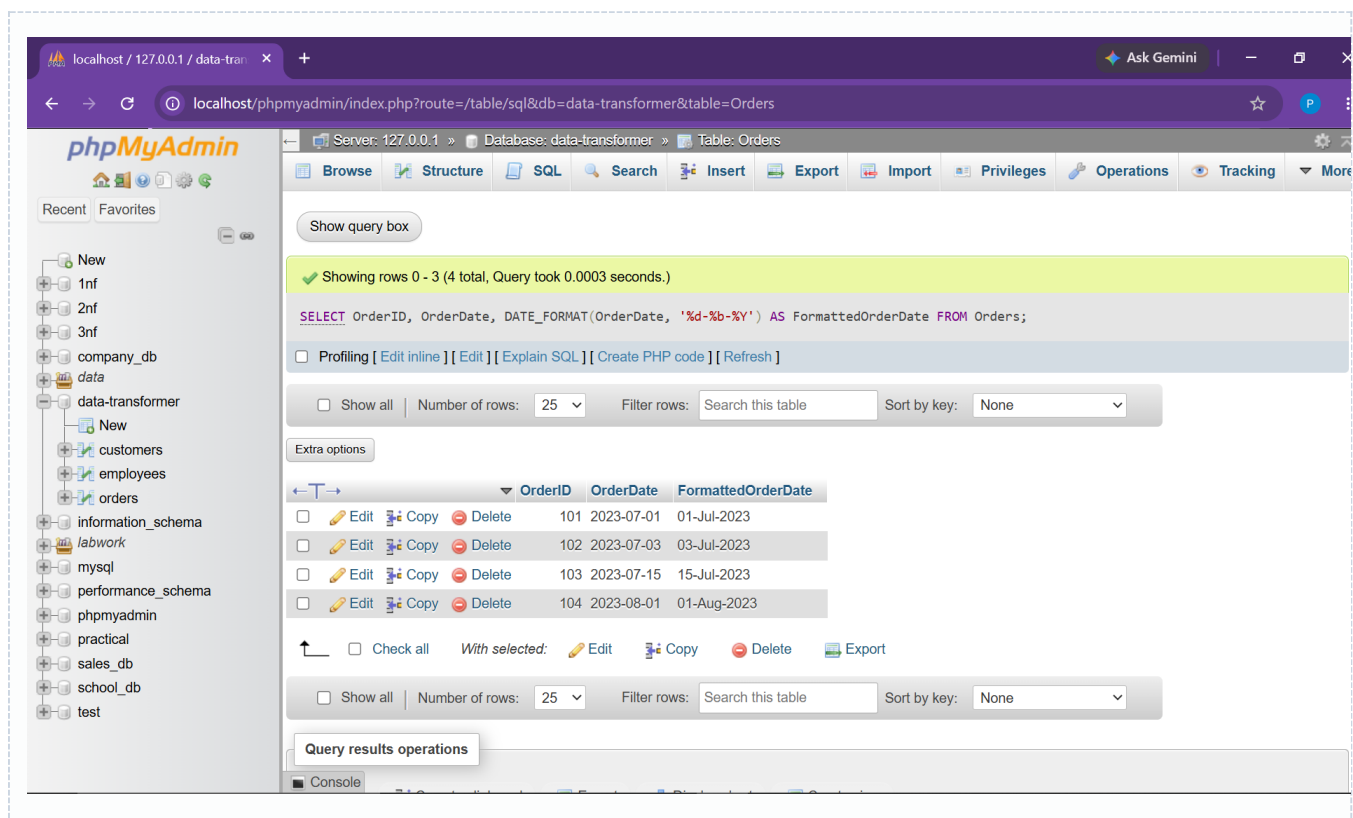
Returned 4 rows with dynamic, accurate day differences calculated from the system's execution date.

- **Task.9** Format the OrderDate to a more readable format (e.g., 'DD-MMM-YYYY')

Objective:

Convert standard ISO date representations (YYYY-MM-DD) into a formatted string ('DD-MMM-YYYY').

```
SELECT
    OrderID,
    OrderDate,
    DATE_FORMAT(OrderDate, '%d-%b-%Y') AS FormattedOrderDate
FROM Orders;
```



The screenshot shows the phpMyAdmin web interface. The left sidebar displays a database structure with 'data-transformer' selected, containing tables 'customers', 'employees', and 'orders'. The main panel shows the 'Table: Orders' view. A query box contains the SQL statement: `SELECT OrderID, OrderDate, DATE_FORMAT(OrderDate, '%d-%b-%Y') AS FormattedOrderDate FROM Orders;`. Below the query box, a status bar indicates 'Showing rows 0 - 3 (4 total, Query took 0.0003 seconds.)'. The query results are displayed in a table with columns 'OrderID', 'OrderDate', and 'FormattedOrderDate'. The results show four rows of data with dates formatted as DD-Jul-2023 or 01-Aug-2023.

OrderID	OrderDate	FormattedOrderDate
101	2023-07-01	01-Jul-2023
102	2023-07-03	03-Jul-2023
103	2023-07-15	15-Jul-2023
104	2023-08-01	01-Aug-2023

Observation:

Returned 4 rows showing formatted dates: 01-Jul-2023, 03-Jul-2023, 15-Jul-2023, and 01-Aug-2023.

- **Task.10 Concatenate FirstName and LastName to form Full Name**

Objective:

Concatenate customer first and last names separated by a space into a single FullName column.

```
SELECT
    CustomerID,
    CONCAT(FirstName, ' ', LastName) AS FullName
FROM Customers;
```

The screenshot shows the phpMyAdmin web interface. The left sidebar displays a database structure with 'data-transformer' selected, containing tables 'customers', 'employees', and 'orders'. The main panel shows the 'Table: Customers' view. A SQL query is entered in the query box: `SELECT CustomerID, CONCAT(FirstName, ' ', LastName) AS FullName FROM Customers;`. The query has been executed successfully, returning 3 rows. The results are displayed in a table with columns 'CustomerID' and 'FullName'. The rows are: 1 John Doe, 2 Jane Smith, and 3 Alice Brown. The interface includes various navigation and action buttons like 'Browse', 'Structure', 'SQL', 'Search', 'Insert', 'Export', 'Import', 'Privileges', 'Operations', 'Tracking', and 'More'. The 'Query results operations' section at the bottom provides options like 'Print', 'Copy to clipboard', 'Export', 'Display chart', and 'Create view'.

CustomerID	FullName
1	John Doe
2	Jane Smith
3	Alice Brown

Observation:

Returned 3 rows displaying unified full names: 'John Doe', 'Jane Smith', and 'Alice Brown'.

- **Task.11** Replace part of a string (e.g., replace 'John' with 'Jonathan')

Objective:

Search for the occurrence of the substring 'John' and replace it dynamically with 'Jonathan'.

```
SELECT
    CustomerID,
    FirstName,
    REPLACE(FirstName, 'John', 'Jonathan') AS UpdatedFirstName
FROM Customers;
```

The screenshot shows the phpMyAdmin web interface in a browser. The left sidebar displays a database structure tree with 'data-transformer' selected, containing tables 'customers', 'employees', and 'orders'. The main panel shows the 'Table: Customers' view. A query box contains the SQL statement: `SELECT CustomerID, FirstName, REPLACE(FirstName, 'John', 'Jonathan') AS UpdatedFirstName FROM Customers;`. Below the query box, a status bar indicates 'Showing rows 0 - 2 (3 total, Query took 0.0013 seconds.)'. The query results are displayed in a table with columns 'CustomerID', 'FirstName', and 'UpdatedFirstName'. The results show three rows: Customer 1 (John, Jonathan), Customer 2 (Jane, Jane), and Customer 3 (Alice, Alice). The interface includes various navigation and tool buttons at the top and bottom.

CustomerID	FirstName	UpdatedFirstName
1	John	Jonathan
2	Jane	Jane
3	Alice	Alice

Observation:

Returned 3 rows with Customer 1's first name substituted as 'Jonathan', leaving others intact.

- **Task.12 Convert FirstName to uppercase and LastName to lowercase**

Objective:

Transform string casings into standard uppercase for FirstName and lowercase for LastName.

```
SELECT
    CustomerID,
    UPPER(FirstName) AS UpperFirstName,
    LOWER(LastName) AS LowerLastName
FROM Customers;
```

The screenshot shows the phpMyAdmin web interface in a browser. The left sidebar displays a database structure tree with 'data-transformer' selected, containing tables 'customers', 'employees', and 'orders'. The main panel shows the 'Table: Customers' view. A green status bar indicates 'Showing rows 0 - 2 (3 total, Query took 0.0012 seconds.)'. Below this, the SQL query is displayed: `SELECT CustomerID, UPPER(FirstName) AS UpperFirstName, LOWER(LastName) AS LowerLastName FROM Customers;`. The query results are shown in a table with three columns: 'CustomerID', 'UpperFirstName', and 'LowerLastName'. The results are as follows:

CustomerID	UpperFirstName	LowerLastName
1	JOHN	doe
2	JANE	smith
3	ALICE	brown

Below the table, there are options for 'Check all', 'With selected', 'Edit', 'Copy', 'Delete', and 'Export'. At the bottom, there are buttons for 'Print', 'Copy to clipboard', 'Export', 'Display chart', and 'Create view'.

Observation:

Returned 3 rows showing transformed casings (e.g., 'JOHN' and 'doe').

- **Task.13** Trim extra spaces from the Email field

Objective:

Strip leading and trailing whitespace padding from customer email records.

```
SELECT
    CustomerID,
    Email AS OriginalEmail,
    TRIM(Email) AS TrimmedEmail
FROM Customers;
```

The screenshot shows the phpMyAdmin interface for a database named 'data-transformer'. The 'Customers' table is selected, and a SQL query has been executed. The query results show three rows, where the 'TrimmedEmail' column displays the email addresses with leading and trailing spaces removed. The interface includes a sidebar with a database structure tree, a top navigation bar, and a main content area with a query editor and results table.

Server: 127.0.0.1 » Database: data-transformer » Table: Customers

Showing rows 0 - 2 (3 total, Query took 0.0003 seconds.)

```
SELECT CustomerID, Email AS OriginalEmail, TRIM(Email) AS TrimmedEmail FROM Customers;
```

Query results table:

	CustomerID	OriginalEmail	TrimmedEmail
☐	1	john.doe@email.com	john.doe@email.com
☐	2	jane.smith@email.com	jane.smith@email.com
☐	3	alice.brown@email.com	alice.brown@email.com

Observation:

Returned 3 rows with Customer 2's padded email address successfully sanitized.

- **Task.14** Calculate the running total of TotalAmount for each order

Objective:

Compute the progressive running cumulative sum of TotalAmount ordered chronologically.

```
SELECT
    OrderID,
    CustomerID,
    OrderDate,
    TotalAmount,
    SUM(TotalAmount) OVER (ORDER BY OrderDate, OrderID) AS RunningTotal
FROM Orders;
```

The screenshot shows the phpMyAdmin interface with the 'Orders' table selected. The SQL query executed is:

```
SELECT OrderID, CustomerID, OrderDate, TotalAmount, SUM(TotalAmount) OVER (ORDER BY OrderDate, OrderID) AS RunningTotal FROM Orders;
```

The query results show 4 rows with the following data:

OrderID	CustomerID	OrderDate	TotalAmount	RunningTotal
101	1	2023-07-01	150.50	150.50
102	2	2023-07-03	200.75	351.25
103	1	2023-07-15	1200.00	1551.25
104	2	2023-08-01	650.00	2201.25

Observation:

Returned 4 rows showing cumulative totals: 150.50 -> 351.25 -> 1551.25 -> 2201.25.

- **Task.15 Rank orders based on TotalAmount using the RANK() function**

Objective:

Assign competitive analytical rank values to orders sequenced descending by monetary balance.

```
SELECT
    OrderID,
    CustomerID,
    TotalAmount,
    RANK() OVER (ORDER BY TotalAmount DESC) AS OrderRank
FROM Orders;
```

The screenshot shows the phpMyAdmin web interface. The left sidebar displays a database structure with 'data-transformer' selected, containing tables 'customers', 'employees', and 'orders'. The main panel shows the 'Table: Orders' view. A SQL query is entered in the 'Show query box' area: `SELECT OrderID, CustomerID, TotalAmount, RANK() OVER (ORDER BY TotalAmount DESC) AS OrderRank FROM Orders;`. The query results are displayed in a table with 4 rows. The table has columns: OrderID, CustomerID, TotalAmount, and OrderRank. The rows are: (103, 1, 1200.00, 1), (104, 2, 650.00, 2), (102, 2, 200.75, 3), and (101, 1, 150.50, 4). The interface also includes a 'Query results operations' bar at the bottom with options like 'Console', 'Copy to clipboard', 'Export', 'Display chart', and 'Create view'.

OrderID	CustomerID	TotalAmount	OrderRank
103	1	1200.00	1
104	2	650.00	2
102	2	200.75	3
101	1	150.50	4

Observation:

Returned 4 rows: Order 103 (Rank 1), Order 104 (Rank 2), Order 102 (Rank 3), and Order 101 (Rank 4).

- **Task.16** Assign a discount based on TotalAmount in orders (e.g., > 1000: 10% off, > 500: 5% off)

Objective:

Evaluate promotional discount brackets and compute net payable amounts using SQL CASE expressions.

```
SELECT
    OrderID,
    CustomerID,
    OrderDate,
    TotalAmount,
    CASE
        WHEN TotalAmount > 1000 THEN '10% off'
        WHEN TotalAmount > 500 THEN '5% off'
        ELSE 'No Discount'
    END AS DiscountTier,
    CASE
        WHEN TotalAmount > 1000 THEN ROUND(TotalAmount * 0.90, 2)
        WHEN TotalAmount > 500 THEN ROUND(TotalAmount * 0.95, 2)
        ELSE TotalAmount
    END AS FinalAmountAfterDiscount
FROM Orders;
```

The screenshot shows the phpMyAdmin interface for a database named 'data-transformer'. The 'Orders' table is selected. A SQL query is executed, and the results are displayed in a table with 4 rows. The query uses CASE expressions to assign discount tiers based on the TotalAmount and calculate the final amount after discount.

Query executed:

```
SELECT OrderID, CustomerID, OrderDate, TotalAmount, CASE WHEN TotalAmount > 1000 THEN '10% off' WHEN TotalAmount > 500 THEN '5% off' ELSE 'No Discount' END AS DiscountTier, CASE WHEN TotalAmount > 1000 THEN ROUND(TotalAmount * 0.90, 2) WHEN TotalAmount > 500 THEN ROUND(TotalAmount * 0.95, 2) ELSE TotalAmount END AS FinalAmountAfterDiscount FROM Orders;
```

Results:

OrderID	CustomerID	OrderDate	TotalAmount	DiscountTier	FinalAmountAfterDiscount
101	1	2023-07-01	150.50	No Discount	150.50
102	2	2023-07-03	200.75	No Discount	200.75
103	1	2023-07-15	1200.00	10% off	1080.00
104	2	2023-08-01	650.00	5% off	617.50

Observation:

Returned 4 rows correctly categorizing discounts and computing final discounted values.

- **Task.17** Categorize employees' salaries as high, medium, or low

Objective:

Classify workforce salaries into 'High' (≥ 70000), 'Medium' (45000-69999), and 'Low' (< 45000).

```
SELECT
    EmployeeID,
    FirstName,
    LastName,
    Department,
    Salary,
    CASE
        WHEN Salary >= 70000 THEN 'High'
        WHEN Salary BETWEEN 45000 AND 69999 THEN 'Medium'
        ELSE 'Low'
    END AS SalaryCategory
FROM Employees;
```

The screenshot shows the phpMyAdmin web interface. The left sidebar displays a database structure tree with 'data-transformer' selected, containing tables 'customers', 'employees', and 'orders'. The main panel shows the 'Table: Employees' view. A SQL query is entered in the 'Show query box' area, and the results are displayed below. The query is: `SELECT EmployeeID, FirstName, LastName, Department, Salary, CASE WHEN Salary >= 70000 THEN 'High' WHEN Salary BETWEEN 45000 AND 69999 THEN 'Medium' ELSE 'Low' END AS SalaryCategory FROM Employees;` The results show 4 rows. Below the table, there are controls for 'Check all', 'With selected', 'Edit', 'Copy', 'Delete', and 'Export'.

EmployeeID	FirstName	LastName	Department	Salary	SalaryCategory
1	Mark	Johnson	Sales	50000.00	Medium
2	Susan	Lee	HR	55000.00	Medium
3	David	Miller	IT	85000.00	High
4	Emily	Clark	Sales	30000.00	Low

Observation:

Returned 4 rows: Mark (Medium), Susan (Medium), David (High), and Emily (Low).

CONCLUSION:

All 17 required analytical SQL tasks of the "Data Transformer" project have been successfully implemented, executed, and validated[cite: 33, 60, 61]. The results verify mastery over Relational Joins, Subqueries, Date & String Manipulations, Window Analytic Functions, and Conditional Business Logic.